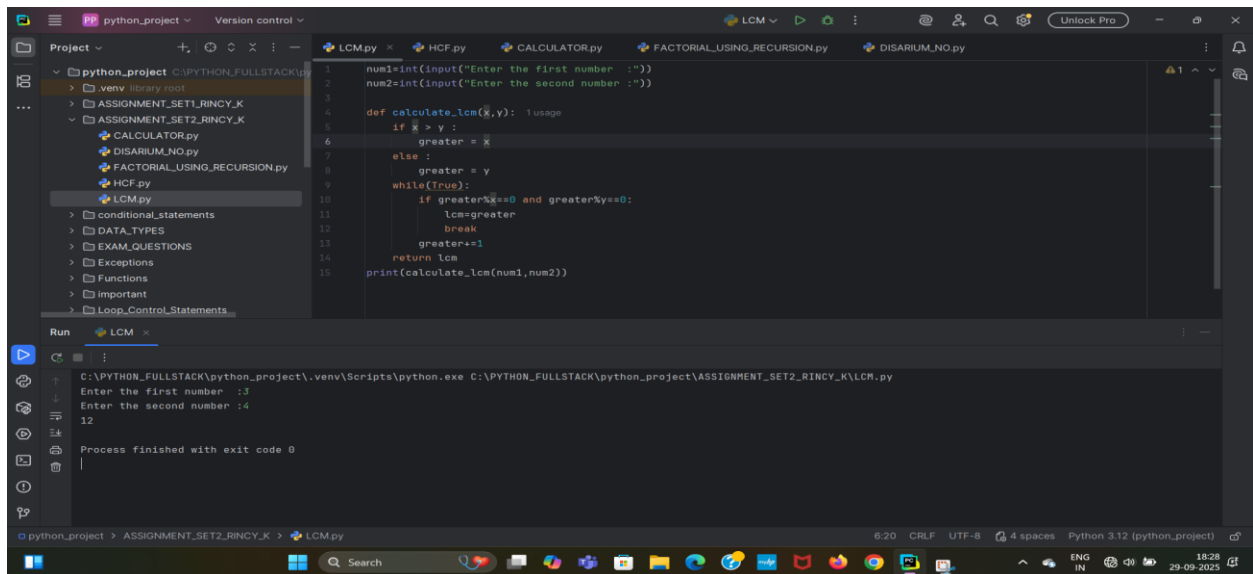


2. PYTHON PROGRAMS

RINCY K (AUG2025)

1.LCM



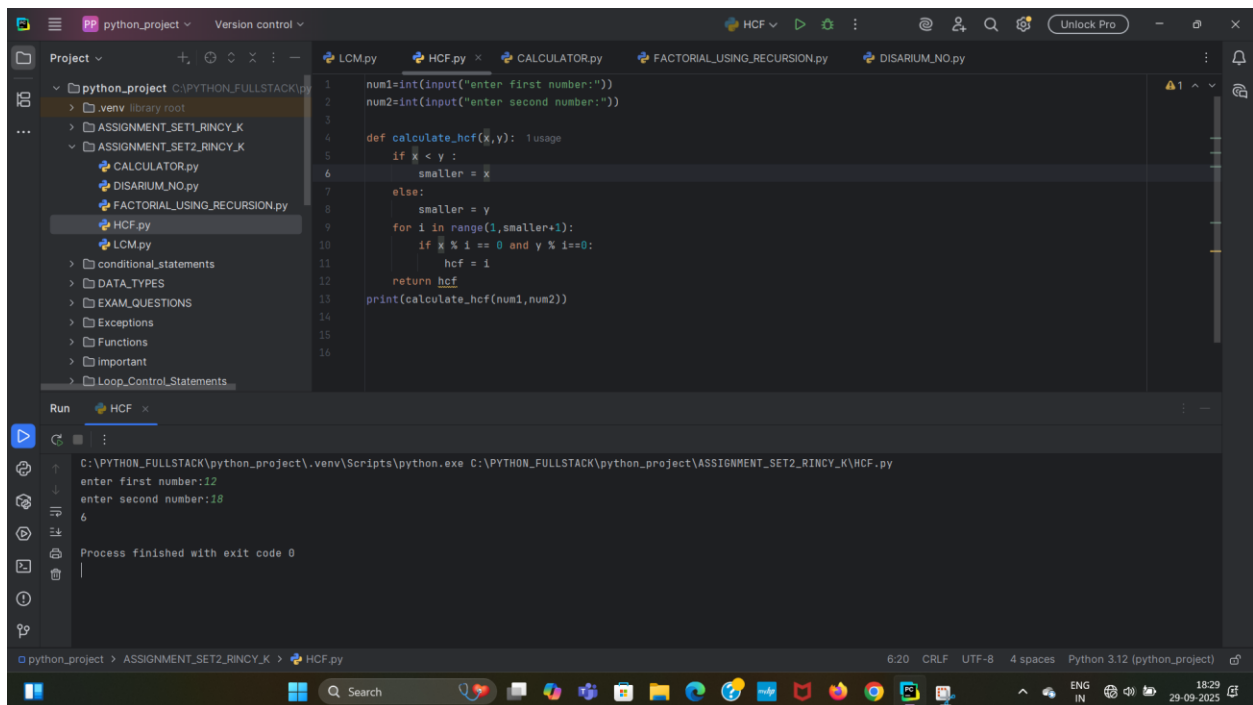
The screenshot shows a Python IDE with a project named 'python_project'. The file explorer on the left shows a directory structure with files like 'venv', 'ASSIGNMENT_SET1_RINCY_K', 'ASSIGNMENT_SET2_RINCY_K', 'CALCULATOR.py', 'DISARIUM_NO.py', 'FACTORIAL_USING_RECURSION.py', 'HCF.py', and 'LCM.py'. The main editor displays the code for 'LCM.py':

```
1 num1=int(input("Enter the first number :"))
2 num2=int(input("Enter the second number :"))
3
4 def calculate_lcm(x,y):
5     if x > y :
6         greater = x
7     else :
8         greater = y
9     while(True):
10         if greater%x==0 and greater%y==0:
11             lcm=greater
12             break
13         greater+=1
14     return lcm
15 print(calculate_lcm(num1,num2))
```

The Run console shows the execution of the program:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\LCM.py
Enter the first number :9
Enter the second number :4
12
Process finished with exit code 0
```

2.HCF



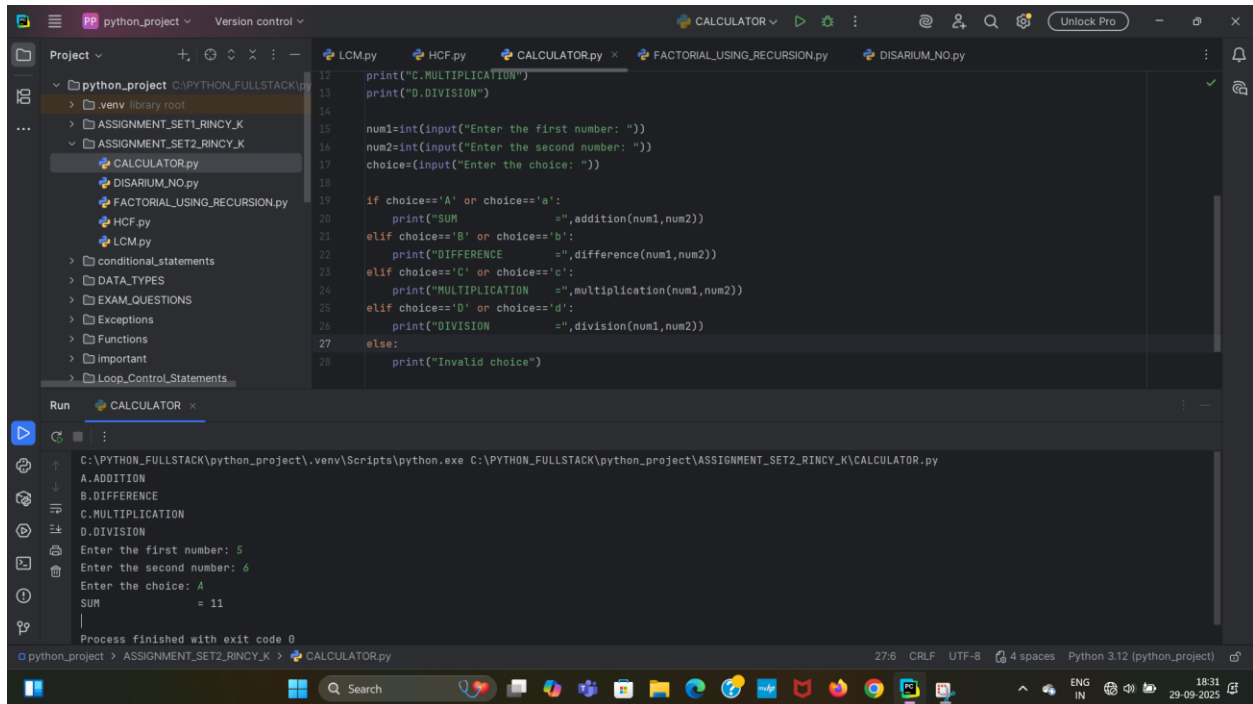
The screenshot shows the same Python IDE with the file explorer highlighting 'HCF.py'. The main editor displays the code for 'HCF.py':

```
1 num1=int(input("enter first number:"))
2 num2=int(input("enter second number:"))
3
4 def calculate_hcf(x,y):
5     if x < y :
6         smaller = x
7     else:
8         smaller = y
9     for i in range(1,smaller+1):
10         if x % i == 0 and y % i==0:
11             hcf = i
12     return hcf
13 print(calculate_hcf(num1,num2))
14
15
16
```

The Run console shows the execution of the program:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\HCF.py
enter first number:12
enter second number:18
6
Process finished with exit code 0
```

3.CALCULATOR



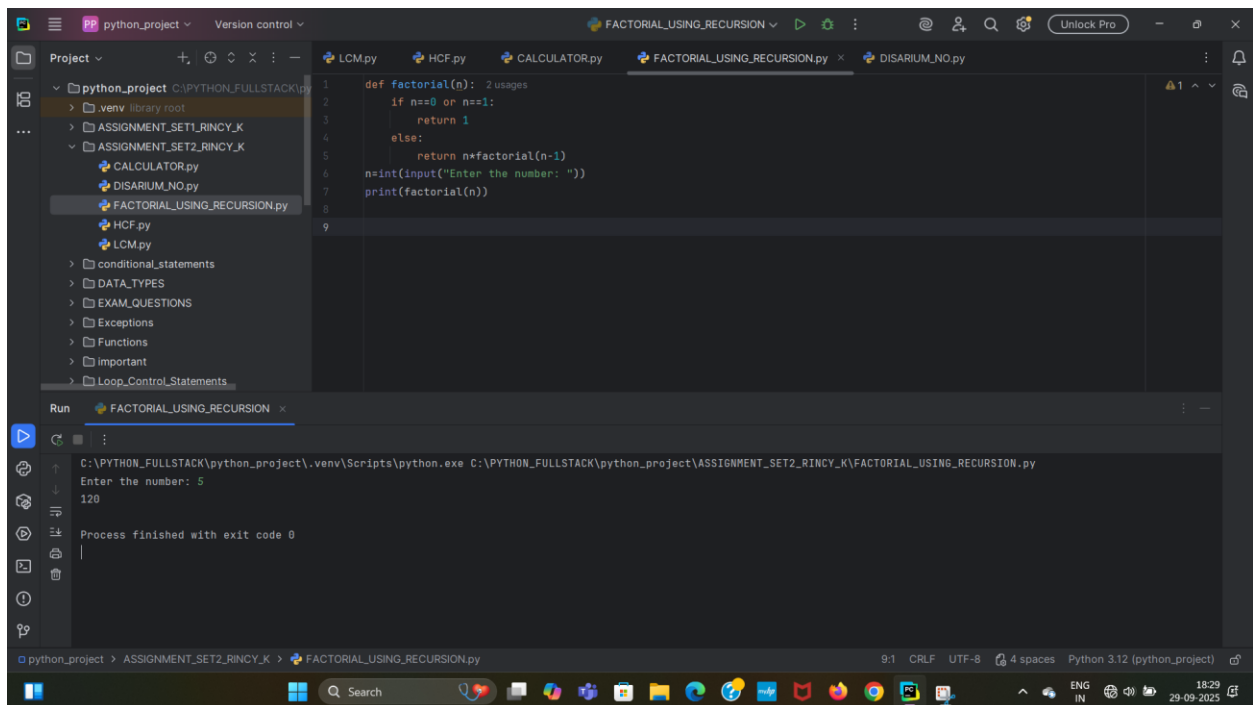
The screenshot shows a Python IDE with a project named 'python_project'. The file explorer on the left shows a directory structure with files like 'venv', 'ASSIGNMENT_SET1_RINCY_K', 'ASSIGNMENT_SET2_RINCY_K', 'CALCULATOR.py', 'DISARIUM_NO.py', 'FACTORIAL_USING_RECURSION.py', 'HCF.py', and 'LCM.py'. The 'CALCULATOR.py' file is open in the editor, showing the following code:

```
12 print("C.MULTIPLICATION")
13 print("D.DIVISION")
14
15 num1=int(input("Enter the first number: "))
16 num2=int(input("Enter the second number: "))
17 choice=(input("Enter the choice: "))
18
19 if choice=='A' or choice=='a':
20     print("SUM      =",addition(num1,num2))
21 elif choice=='B' or choice=='b':
22     print("DIFFERENCE  =",difference(num1,num2))
23 elif choice=='C' or choice=='c':
24     print("MULTIPLICATION  =",multiplication(num1,num2))
25 elif choice=='D' or choice=='d':
26     print("DIVISION      =",division(num1,num2))
27 else:
28     print("Invalid choice")
```

The 'Run' panel at the bottom shows the execution of 'CALCULATOR.py'. The output is as follows:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\CALCULATOR.py
A.ADDITION
B.DIFFERENCE
C.MULTIPLICATION
D.DIVISION
Enter the first number: 5
Enter the second number: 6
Enter the choice: A
SUM      = 11
Process finished with exit code 0
```

4.FACTORIAL(USING RECURSION)



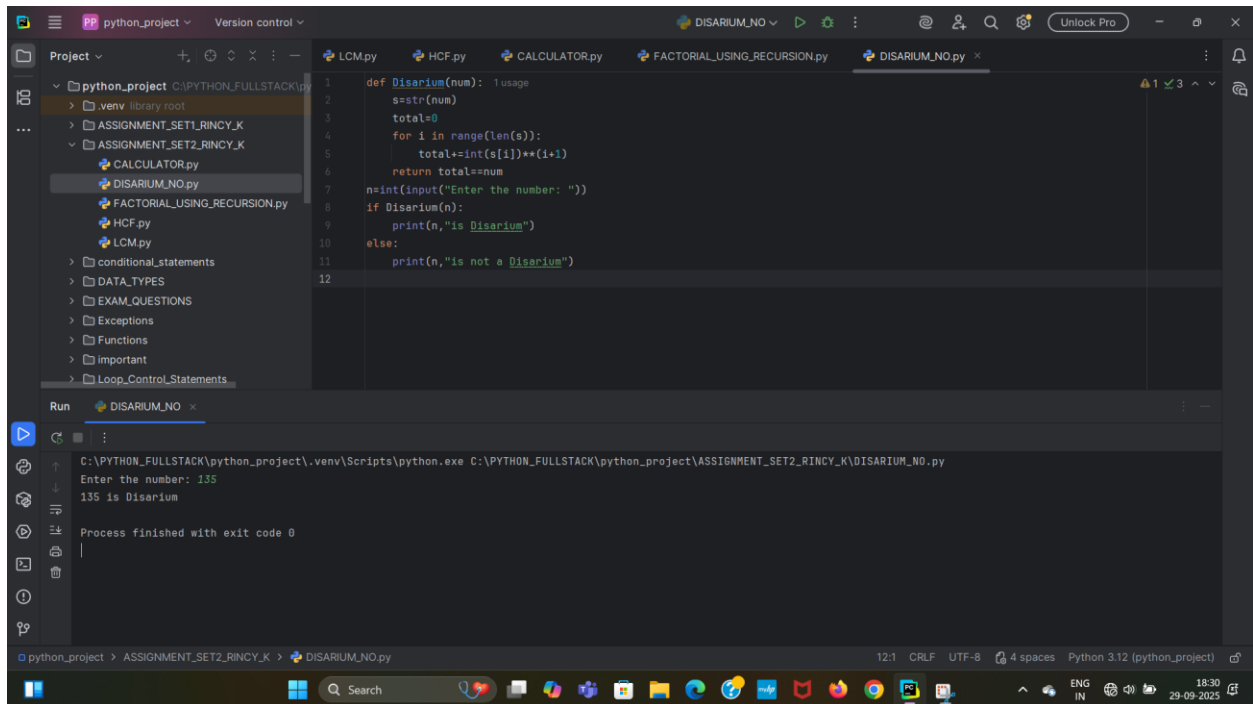
The screenshot shows the same Python IDE with the 'FACTORIAL_USING_RECURSION.py' file open in the editor. The code is as follows:

```
1 def factorial(n): 2 usages
2     if n==0 or n==1:
3         return 1
4     else:
5         return n*factorial(n-1)
6 n=int(input("Enter the number: "))
7 print(factorial(n))
8
9
```

The 'Run' panel at the bottom shows the execution of 'FACTORIAL_USING_RECURSION.py'. The output is as follows:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\FACTORIAL_USING_RECURSION.py
Enter the number: 5
120
Process finished with exit code 0
```

5.DISARIUM NUMBER



```
1 def Disarium(num):
2     s=str(num)
3     total=0
4     for i in range(len(s)):
5         total+=int(s[i])**((i+1))
6     return total==num
7 n=int(input("Enter the number: "))
8 if Disarium(n):
9     print(n,"is Disarium")
10 else:
11     print(n,"is not a Disarium")
12
```

Run HARSHAD_no x

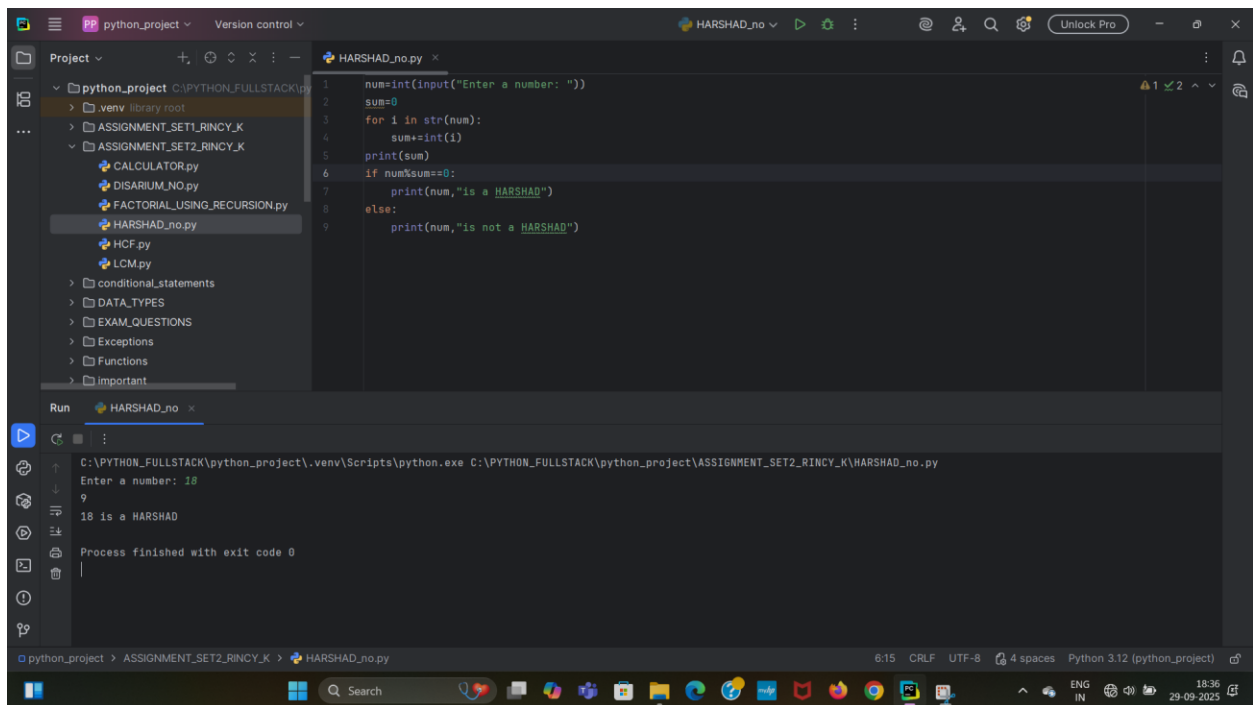
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\DISARIUM_NO.py

Enter the number: 135

135 is Disarium

Process finished with exit code 0

6.HARSHAD NUMBER



```
1 num=int(input("Enter a number: "))
2 sum=0
3 for i in str(num):
4     sum+=int(i)
5 print(sum)
6 if num%sum==0:
7     print(num,"is a HARSHAD")
8 else:
9     print(num,"is not a HARSHAD")
```

Run HARSHAD_no x

C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\HARSHAD_no.py

Enter a number: 18

9

18 is a HARSHAD

Process finished with exit code 0

7.NUMBER OF DIGITS

The screenshot shows a code editor with a project named 'python_project'. The file explorer on the left shows a directory structure with 'python_project' containing 'venv', 'ASSIGNMENT_SET1_RINCY_K', and 'ASSIGNMENT_SET2_RINCY_K'. The 'ASSIGNMENT_SET2_RINCY_K' directory contains several Python files, including 'Number_of digits.py'. The code in 'Number_of digits.py' is as follows:

```
1 #Check whether the number of digits present in an integer
2
3 def count_digits(num):
4     return len(str(abs(num)))
5
6 n=int(input("Enter a number :"))
7 print("Number of digits :",count_digits(n))
```

The 'Run' panel at the bottom shows the execution of the script. The command executed is 'C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe "C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\Number_of digits.py"'. The output is 'Enter a number :14579006' followed by 'Number of digits : 8'. The process finished with exit code 0.

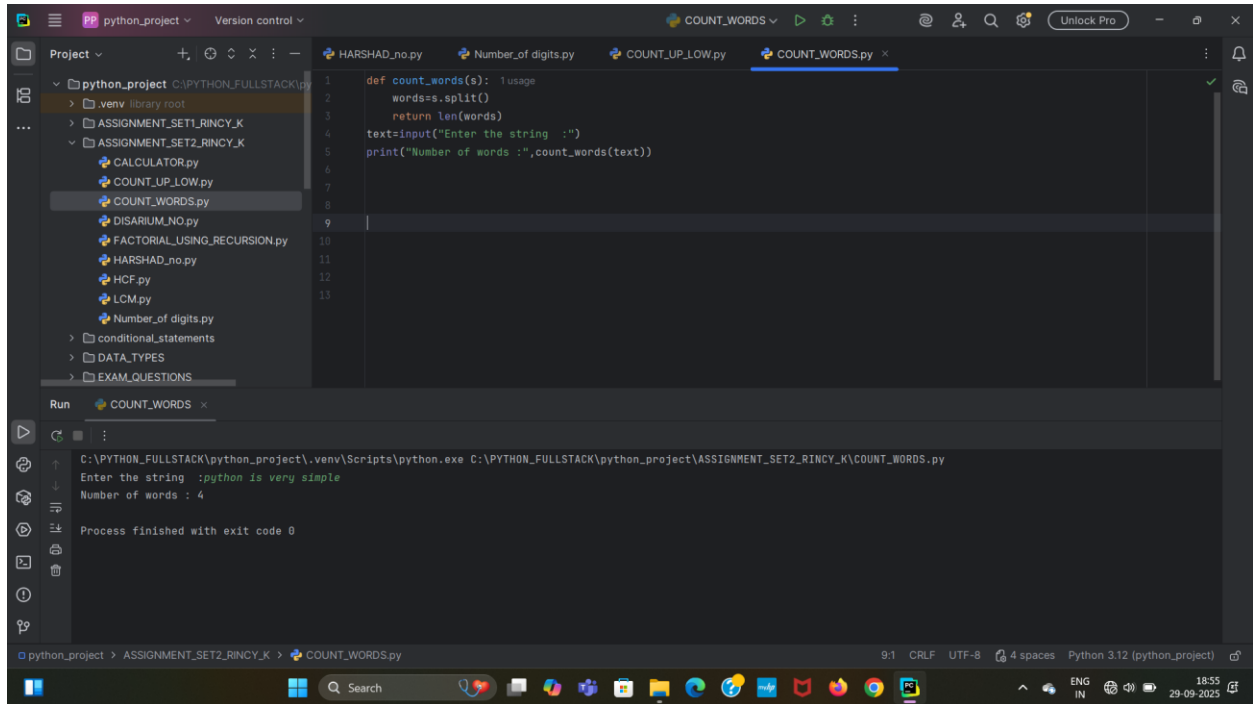
8.COUNT-UPPER CASE & LOWER CASE

The screenshot shows a code editor with a project named 'python_project'. The file explorer on the left shows a directory structure with 'python_project' containing 'venv', 'ASSIGNMENT_SET1_RINCY_K', and 'ASSIGNMENT_SET2_RINCY_K'. The 'ASSIGNMENT_SET2_RINCY_K' directory contains several Python files, including 'COUNT_UP&LOWER.py'. The code in 'COUNT_UP&LOWER.py' is as follows:

```
1 text=input("Enter a string :")
2 upper=0
3 lower=0
4 for char in text:
5     if char.isupper():
6         upper+=1
7     elif char.islower():
8         lower+=1
9
10 print("Upper :",upper)
11 print("Lower :",lower)
12
13
```

The 'Run' panel at the bottom shows the execution of the script. The command executed is 'C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\COUNT_UP&LOWER.py'. The output is 'Enter a string :HeLloWoRLD' followed by 'Upper : 6' and 'Lower : 4'. The process finished with exit code 0.

9.COUNT WORDS IN A STRING



```
1 def count_words(s):
2     words=s.split()
3     return len(words)
4     text=input("Enter the string :")
5     print("Number of words :",count_words(text))
6
7
8
9
10
11
12
13
```

Run: COUNT_WORDS

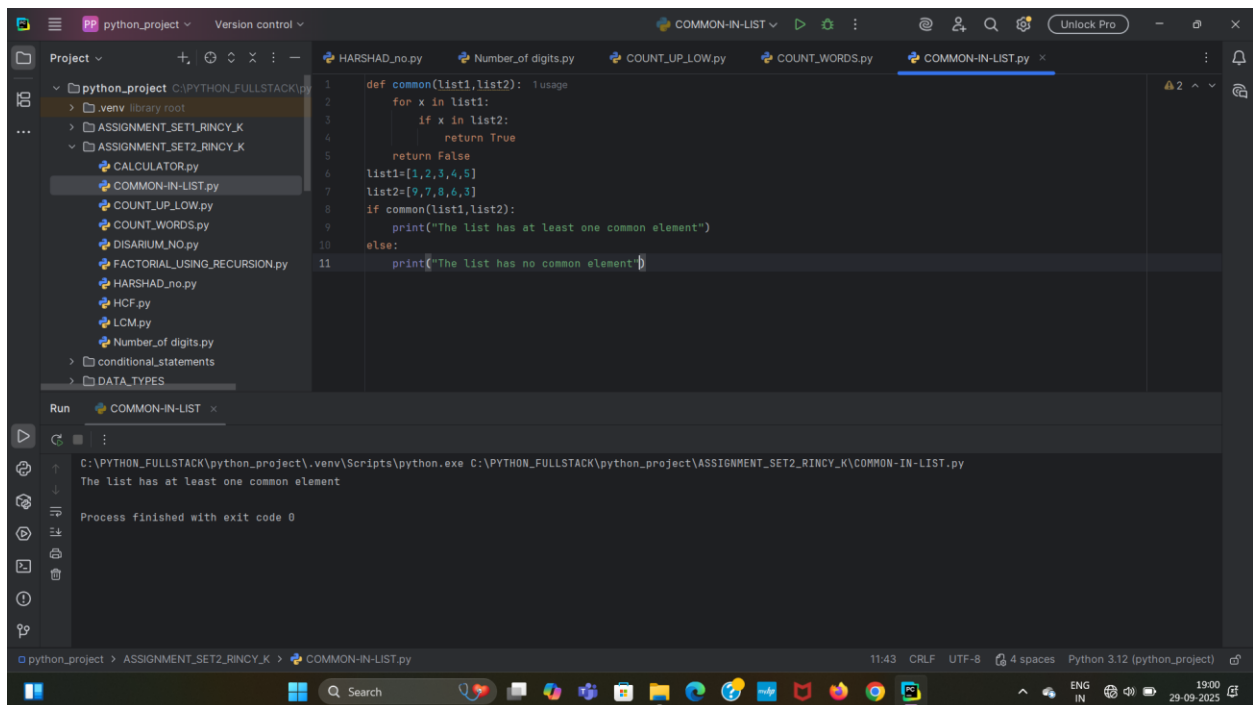
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\COUNT_WORDS.py

Enter the string :python is very simple

Number of words : 4

Process finished with exit code 0

10.COMMON ELEMENTS IN A LIST



```
1 def common(list1,list2):
2     for x in list1:
3         if x in list2:
4             return True
5     return False
6     list1=[1,2,3,4,5]
7     list2=[9,7,8,6,3]
8     if common(list1,list2):
9         print("The list has at least one common element")
10    else:
11        print("The list has no common element")
```

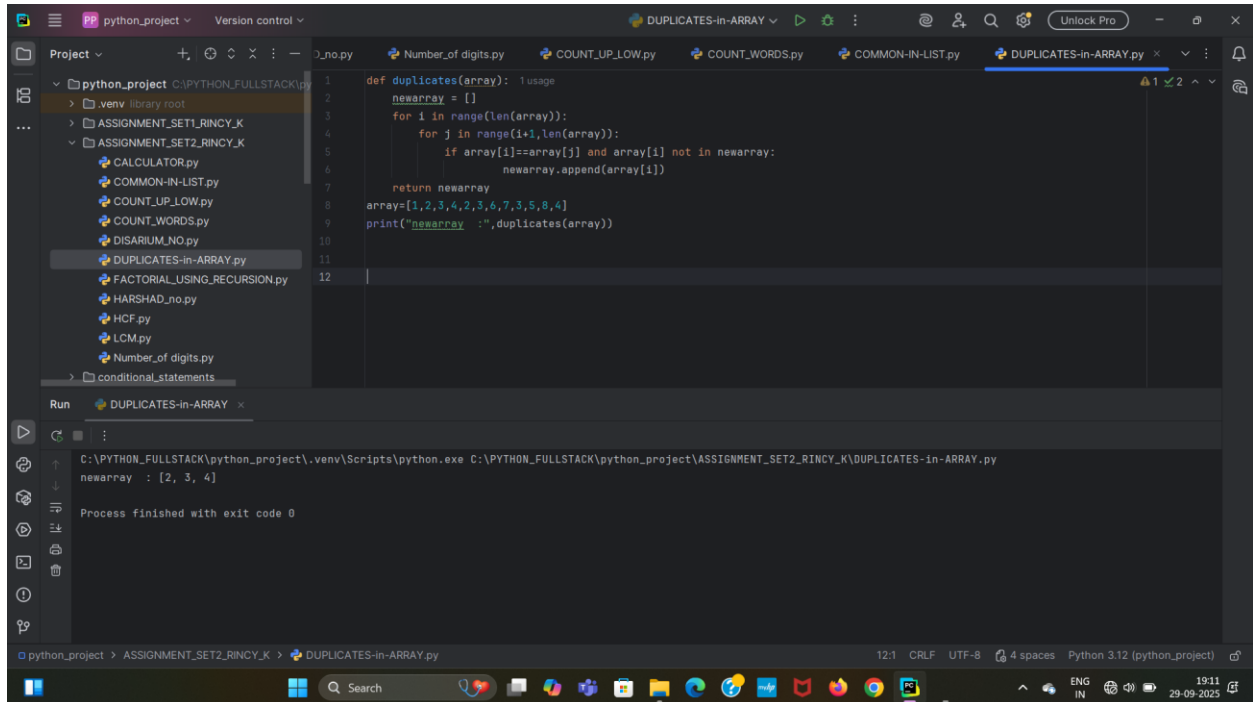
Run: COMMON-IN-LIST

C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\COMMON-IN-LIST.py

The list has at least one common element

Process finished with exit code 0

11.DUPLICATES IN ARRAY

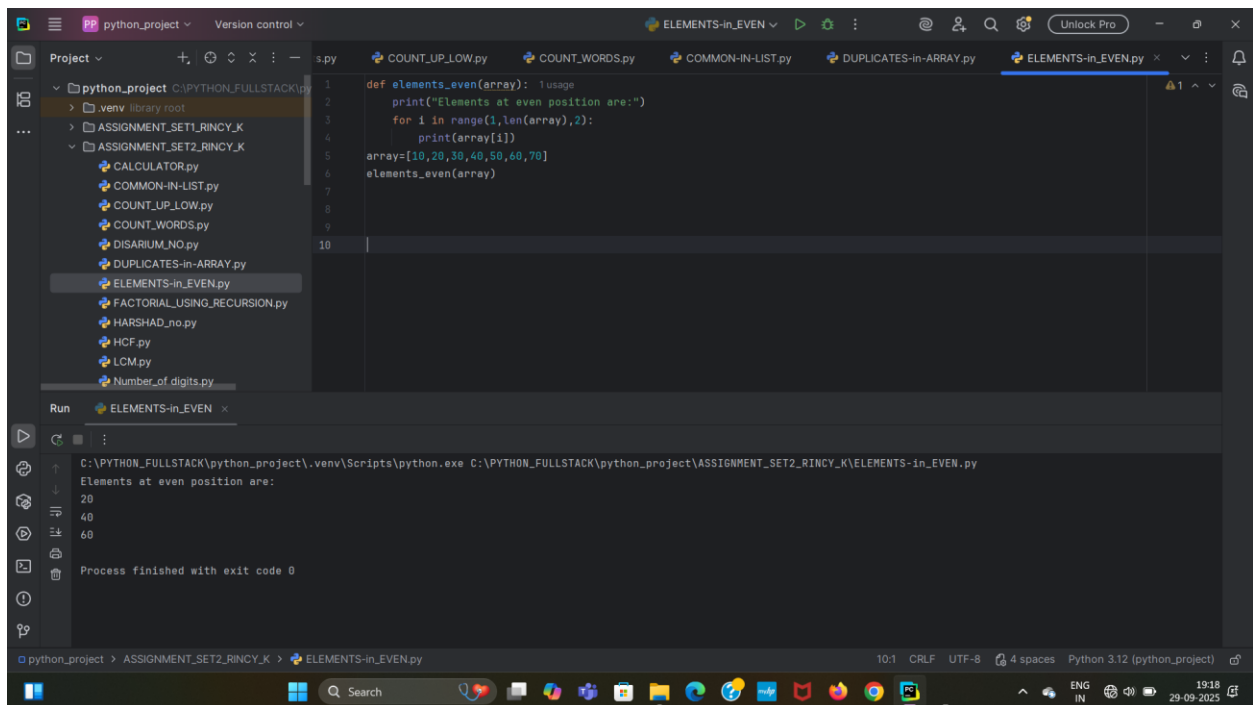


The screenshot shows a Python IDE with a project named 'python_project'. The file explorer on the left shows a directory structure with various Python files. The main editor displays a file named 'DUPLICATES-in-ARRAY.py' with the following code:

```
1 def duplicates(array):
2     newarray = []
3     for i in range(len(array)):
4         for j in range(i+1, len(array)):
5             if array[i] == array[j] and array[i] not in newarray:
6                 newarray.append(array[i])
7     return newarray
8 array = [1, 2, 3, 4, 2, 3, 6, 7, 3, 5, 8, 4]
9 print("newarray :", duplicates(array))
10
11
12
```

The Run console at the bottom shows the output: 'newarray : [2, 3, 4]' and 'Process finished with exit code 0'.

12.ELEMENTS IN EVEN POSITION



The screenshot shows a Python IDE with a project named 'python_project'. The file explorer on the left shows a directory structure with various Python files. The main editor displays a file named 'ELEMENTS-in-EVEN.py' with the following code:

```
1 def elements_even(array):
2     print("Elements at even position are:")
3     for i in range(1, len(array), 2):
4         print(array[i])
5 array = [10, 20, 30, 40, 50, 60, 70]
6 elements_even(array)
7
8
9
10
```

The Run console at the bottom shows the output: 'Elements at even position are:', '20', '40', '60' and 'Process finished with exit code 0'.

13.ELEMENTS IN ODD POSITION

The screenshot shows a VS Code editor with a project named 'python_project'. The file explorer on the left lists several Python files, including 'ELEMENTS-in-ODD.py'. The main editor window displays the code for 'ELEMENTS-in-ODD.py':

```
1 def elements_odd(array): 1 usage
2     print("Elements at odd position are:")
3     for i in range(0,len(array),2):
4         print(array[i])
5     arrays=[10,20,30,40,50,60,70]
6     elements_odd(array)
7
8
9
```

The Run and Debug console at the bottom shows the execution of the script. The output is:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\ELEMENTS-in-ODD.py
Elements at odd position are:
10
30
50
70
Process finished with exit code 0
```

The status bar at the bottom indicates the file is 'ELEMENTS-in-ARRAY.py' (though the editor shows 'ELEMENTS-in-ODD.py'), with settings for 2:43, CRLF, UTF-8, 4 spaces, and Python 3.12.

14.LARGEST

The screenshot shows a VS Code editor with a project named 'python_project'. The file explorer on the left lists several Python files, including 'LARGEST.py'. The main editor window displays the code for 'LARGEST.py':

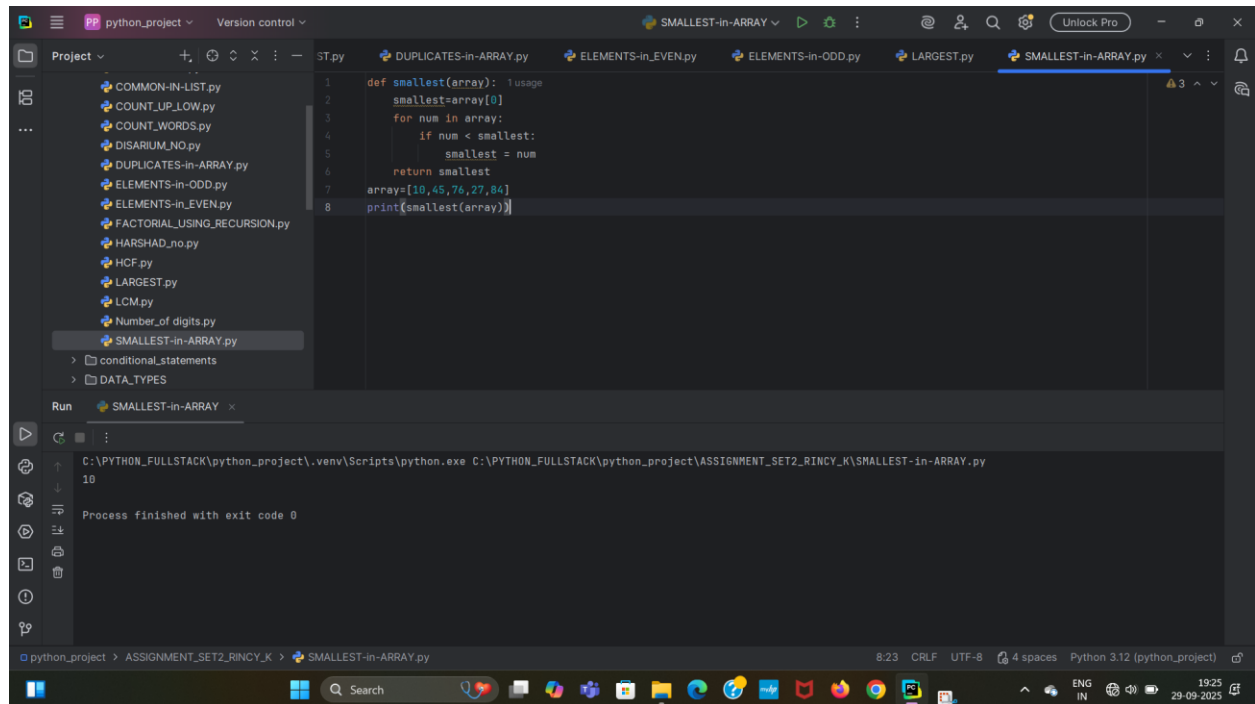
```
1 def largest(array): 1 usage
2     largest=array[0]
3     for num in array:
4         if num > largest:
5             largest = num
6     return largest
7     arrays=[10,45,76,27,84]
8     print(largest(array))
9
```

The Run and Debug console at the bottom shows the execution of the script. The output is:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\LARGEST.py
84
Process finished with exit code 0
```

The status bar at the bottom indicates the file is 'LARGEST.py', with settings for 9:5, CRLF, UTF-8, 4 spaces, and Python 3.12.

15.SMALLEST

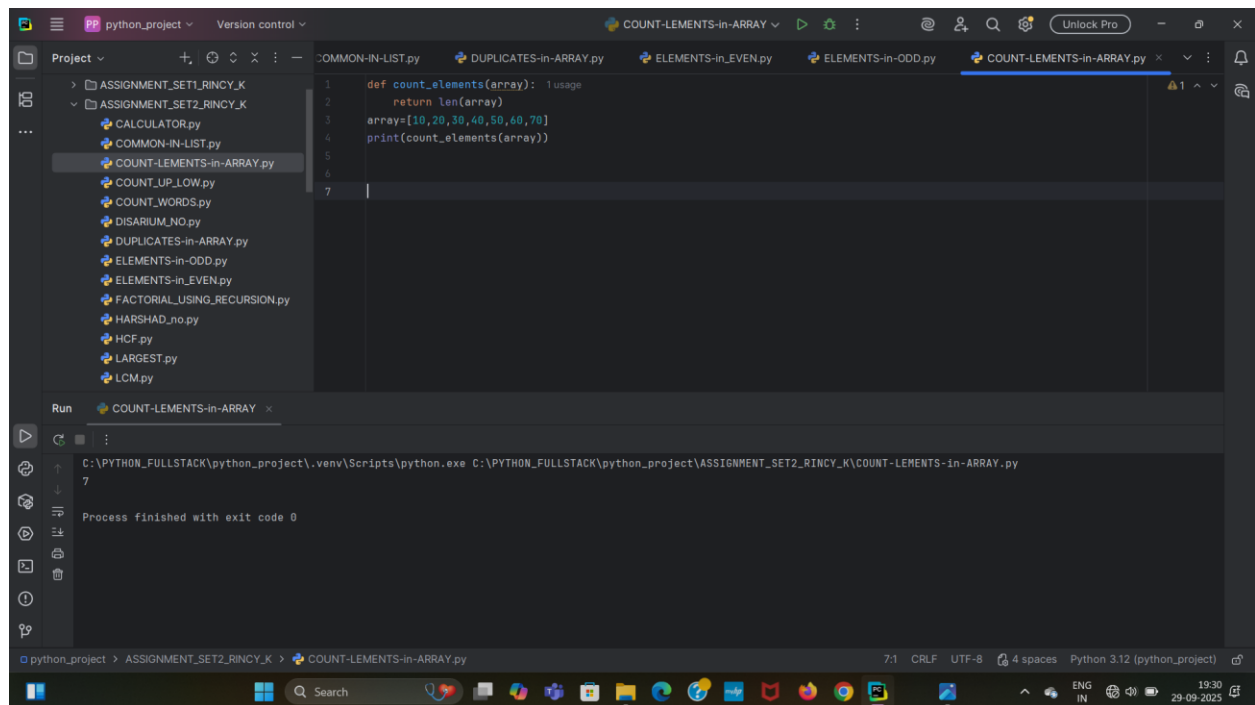


The screenshot shows a VS Code editor window with the file 'SMALLEST-in-ARRAY.py' open. The code defines a function 'smallest' that iterates through an array to find the minimum value. The array is [10, 45, 76, 27, 84]. The output of the program is 10.

```
1 def smallest(array):
2     smallest=array[0]
3     for num in array:
4         if num < smallest:
5             smallest = num
6     return smallest
7 array=[10,45,76,27,84]
8 print(smallest(array))
```

Run: C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\SMALLEST-in-ARRAY.py
10
Process finished with exit code 0

16.COUNT ELEMENTS IN AN ARRAY

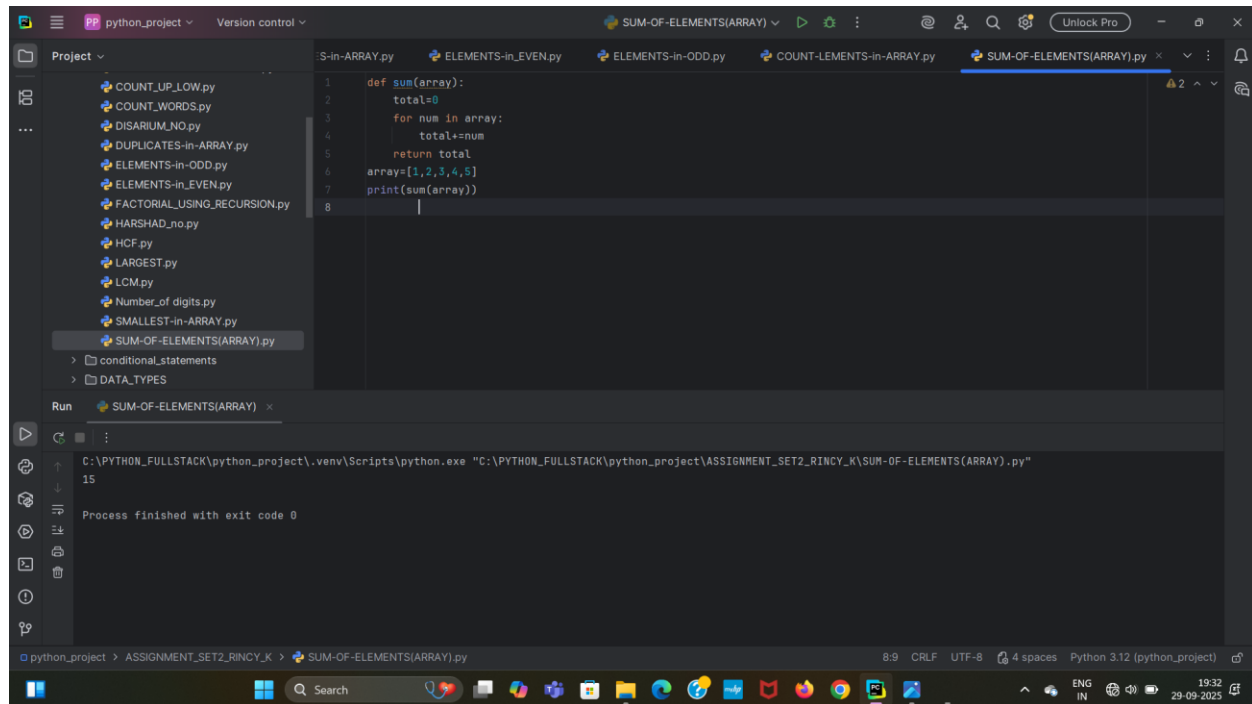


The screenshot shows a VS Code editor window with the file 'COUNT-ELEMENTS-in-ARRAY.py' open. The code defines a function 'count_elements' that returns the length of an array. The array is [10, 20, 30, 40, 50, 60, 70]. The output of the program is 7.

```
1 def count_elements(array):
2     return len(array)
3 array=[10,20,30,40,50,60,70]
4 print(count_elements(array))
5
6
7
```

Run: C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\COUNT-ELEMENTS-in-ARRAY.py
7
Process finished with exit code 0

17.SUM OF ELEMENTS IN AN ARRAY

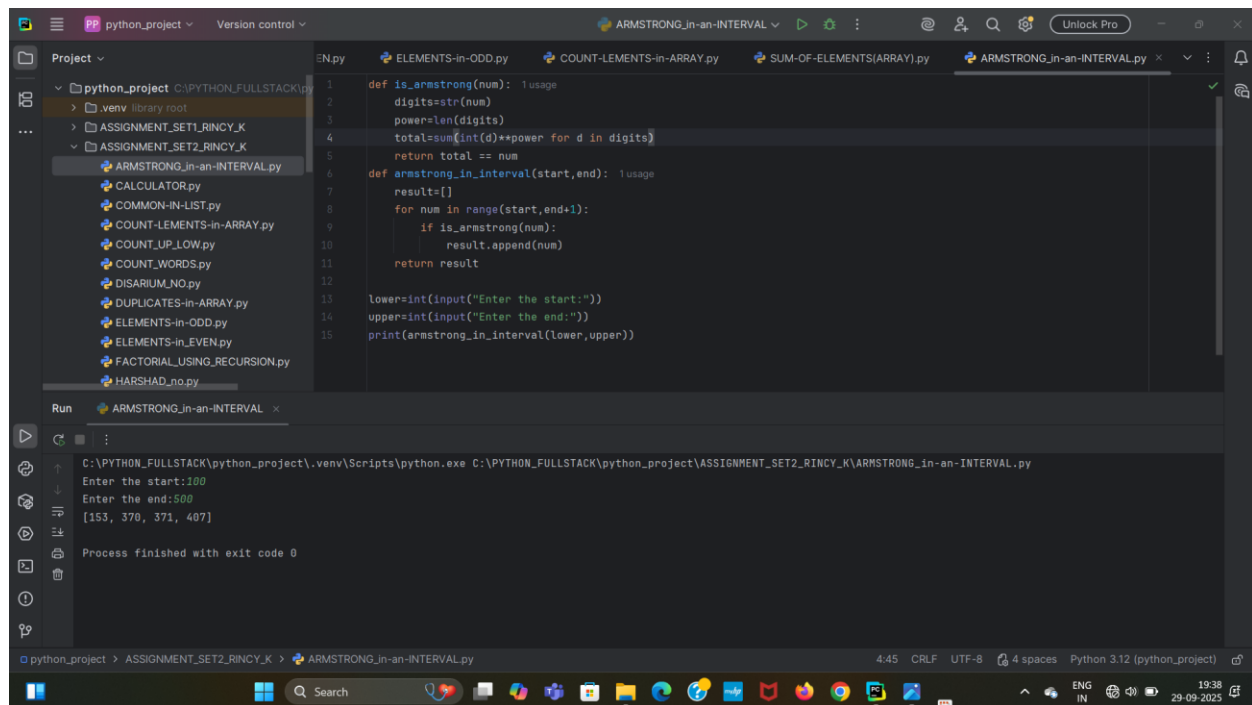


```
1 def sum(array):
2     total=0
3     for num in array:
4         total+=num
5     return total
6 array=[1,2,3,4,5]
7 print(sum(array))
8
```

Run SUM-OF-ELEMENTS(ARRAY) x

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe "C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\SUM-OF-ELEMENTS(ARRAY).py"
15
Process finished with exit code 0
```

18.ARMSTRONG IN AN INTERVAL

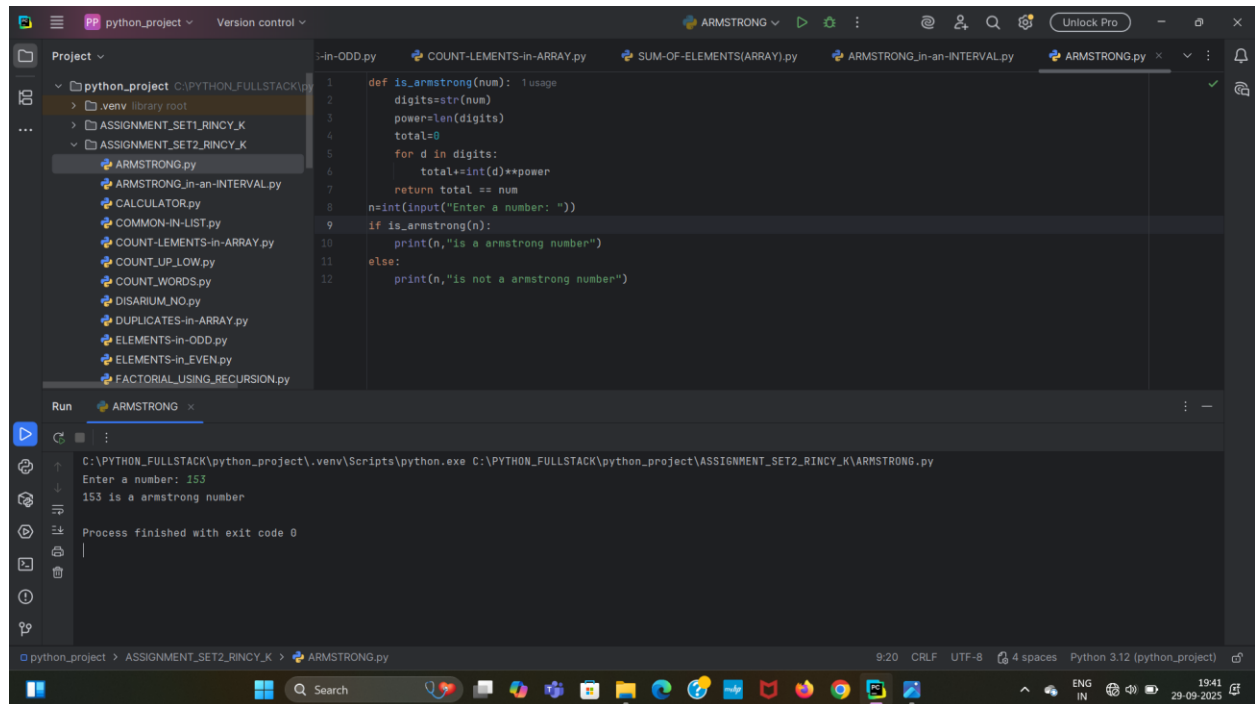


```
1 def is_armstrong(num): 1usage
2     digits=str(num)
3     power=len(digits)
4     total=sum(int(d)**power for d in digits)
5     return total == num
6 def armstrong_in_interval(start,end): 1usage
7     result=[]
8     for num in range(start,end+1):
9         if is_armstrong(num):
10             result.append(num)
11     return result
12
13 lower=int(input("Enter the start:"))
14 upper=int(input("Enter the end:"))
15 print(armstrong_in_interval(lower,upper))
```

Run ARMSTRONG_In-an-INTERVAL x

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\ARMSTRONG_In-an-INTERVAL.py
Enter the start:100
Enter the end:500
[153, 370, 371, 407]
Process finished with exit code 0
```

19.ARMSTRONG

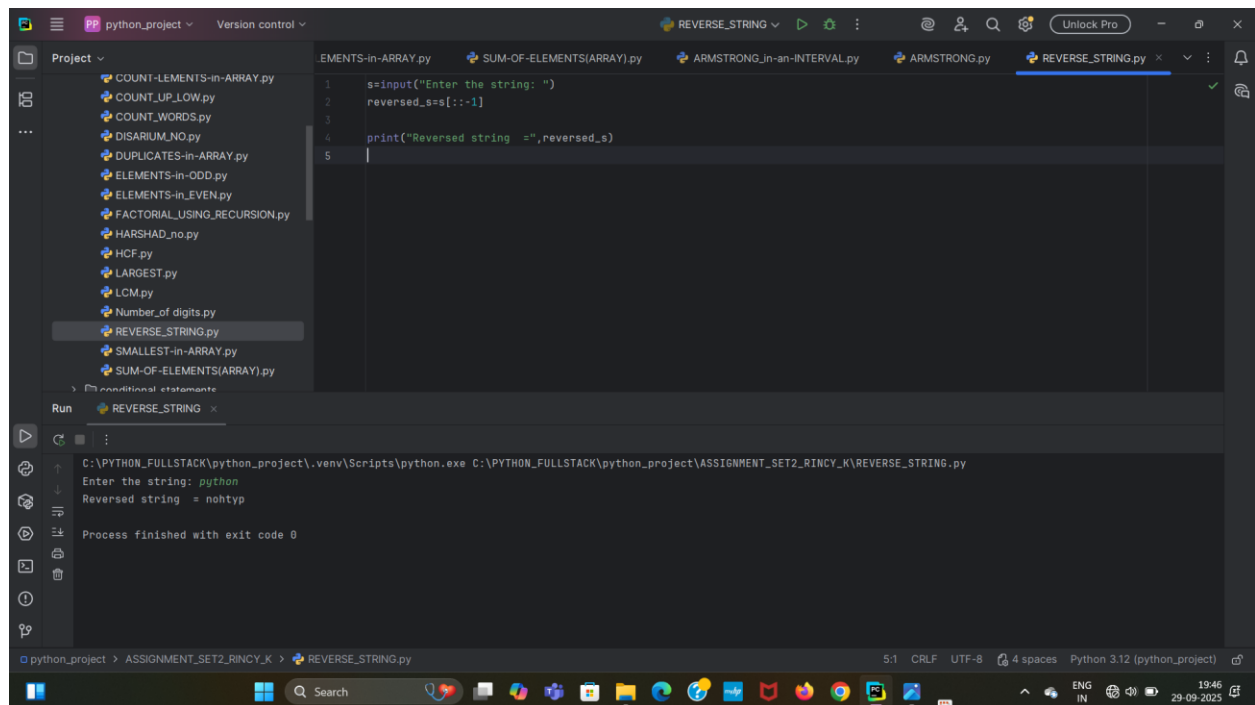


The screenshot shows a code editor with the file `ARMSTRONG.py` open. The code defines a function `is_armstrong(num)` that checks if a number is an Armstrong number. The function converts the number to a string, calculates the sum of each digit raised to the power of the number of digits, and compares it to the original number. The main program prompts the user to enter a number and prints the result.

```
1 def is_armstrong(num):
2     digits=str(num)
3     power=len(digits)
4     total=0
5     for d in digits:
6         total+=int(d)**power
7     return total == num
8 n=int(input("Enter a number: "))
9 if is_armstrong(n):
10    print(n,"is a armstrong number")
11 else:
12    print(n,"is not a armstrong number")
```

The Run window shows the execution of the program. The user entered the number 153, and the program output is "153 is a armstrong number". The process finished with exit code 0.

20.REVERSE STRING

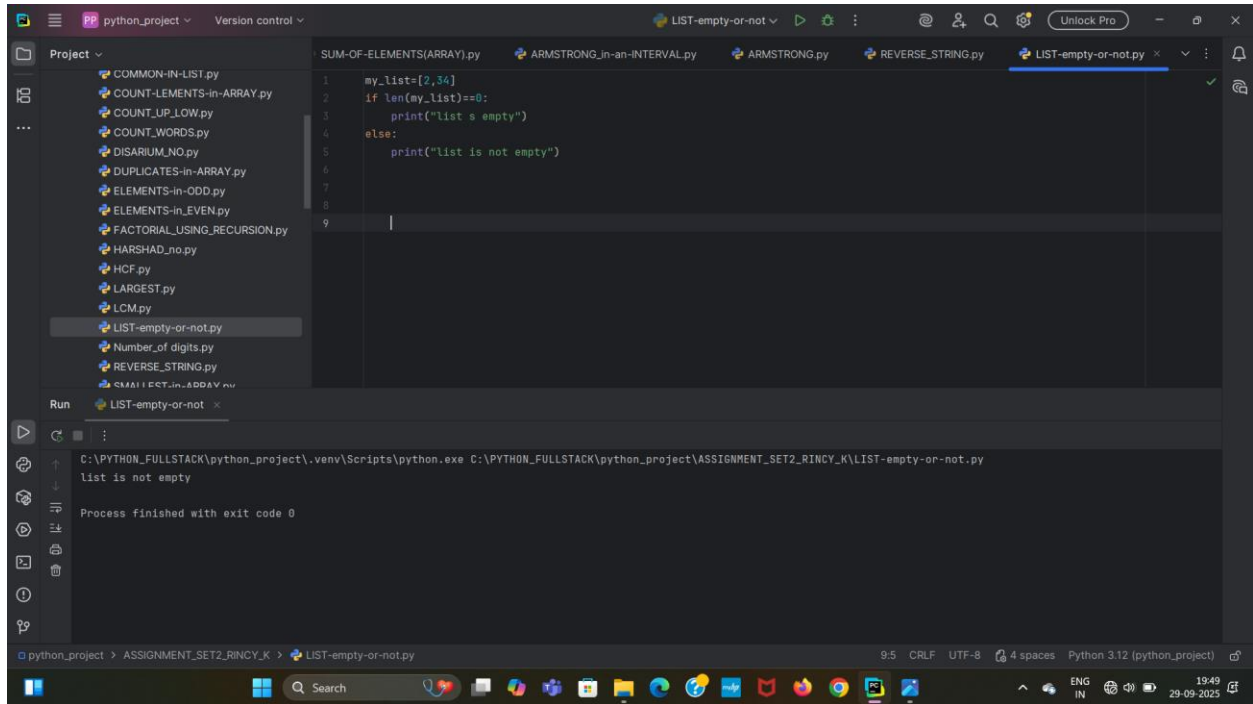


The screenshot shows a code editor with the file `REVERSE_STRING.py` open. The code prompts the user to enter a string and prints the reversed string.

```
1 s=input("Enter the string: ")
2 reversed_s=s[::-1]
3
4 print("Reversed string =",reversed_s)
5
```

The Run window shows the execution of the program. The user entered the string "python", and the program output is "Reversed string = nohtyp". The process finished with exit code 0.

21. LIST EMPTY OR NOT

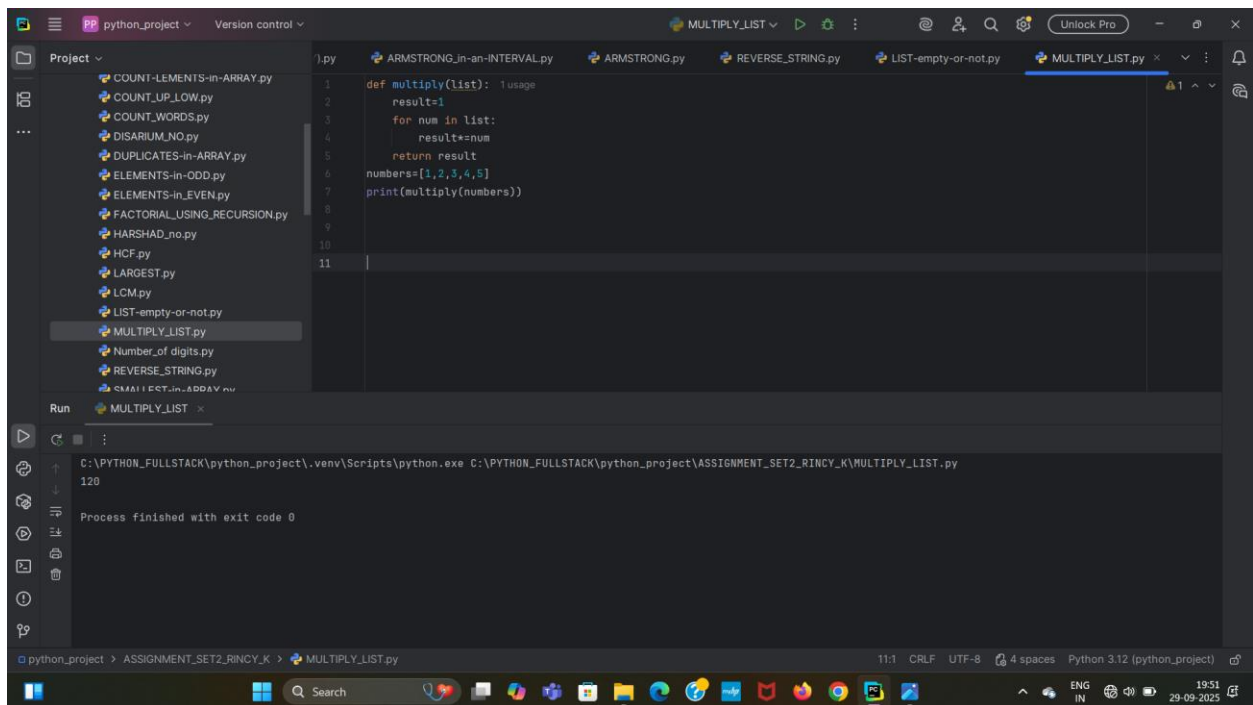


The screenshot shows a code editor with a project named 'python_project'. The file explorer on the left lists various Python files, including 'LIST-empty-or-not.py'. The main editor displays the code for 'LIST-empty-or-not.py':

```
1 my_list=[2,34]
2 if len(my_list)==0:
3     print("list s empty")
4 else:
5     print("List is not empty")
6
7
8
9
```

The Run panel at the bottom shows the command: `C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\LIST-empty-or-not.py`. The output is: `List is not empty`. The process finished with exit code 0.

22. MULTIPLYING ELEMENTS IN A LIST

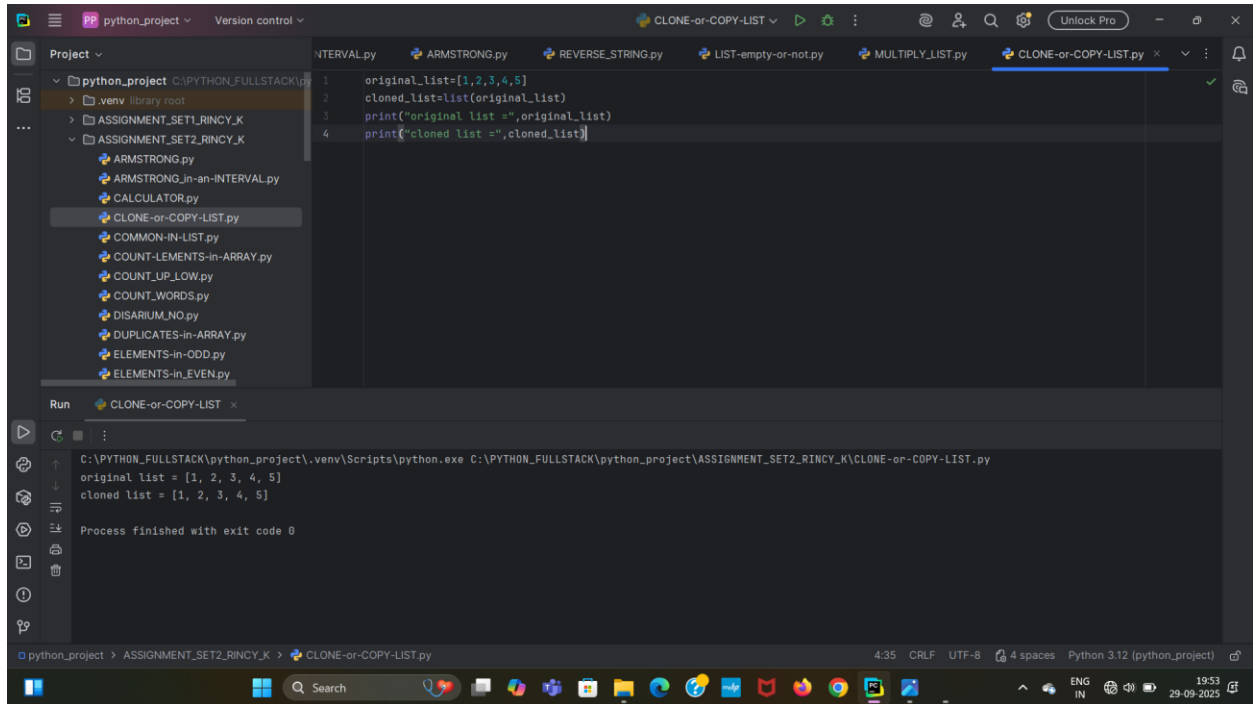


The screenshot shows a code editor with a project named 'python_project'. The file explorer on the left lists various Python files, including 'MULTIPLY_LIST.py'. The main editor displays the code for 'MULTIPLY_LIST.py':

```
1 def multiply(list):
2     result=1
3     for num in list:
4         result*=num
5     return result
6 numbers=[1,2,3,4,5]
7 print(multiply(numbers))
8
9
10
11
```

The Run panel at the bottom shows the command: `C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\MULTIPLY_LIST.py`. The output is: `120`. The process finished with exit code 0.

23.CLONE OR COPY



The screenshot shows a Visual Studio Code editor window with a project named 'python_project'. The file explorer on the left shows a directory structure with various Python files. The main editor displays the code for 'CLONE-or-COPY-LIST.py':

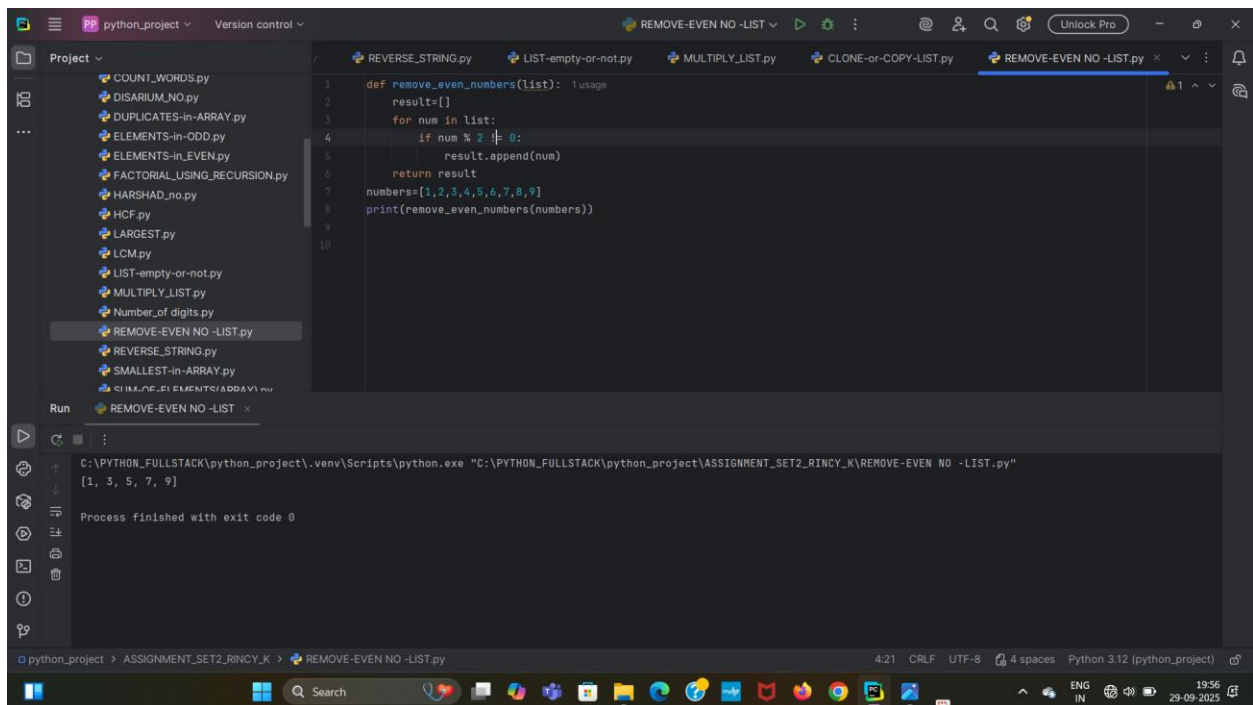
```
1 original_list=[1,2,3,4,5]
2 cloned_list=list(original_list)
3 print("original list =",original_list)
4 print("cloned list =",cloned_list)
```

The Run and Debug console at the bottom shows the execution of the script:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\CLONE-or-COPY-LIST.py
original list = [1, 2, 3, 4, 5]
cloned list = [1, 2, 3, 4, 5]
Process finished with exit code 0
```

The status bar at the bottom indicates the file is 'CLONE-or-COPY-LIST.py' with 4:35 CRLF UTF-8 encoding, 4 spaces, and Python 3.12.

24.REMOVE EVEN NUMBERS



The screenshot shows a Visual Studio Code editor window with a project named 'python_project'. The file explorer on the left shows a directory structure with various Python files. The main editor displays the code for 'REMOVE-EVEN NO -LIST.py':

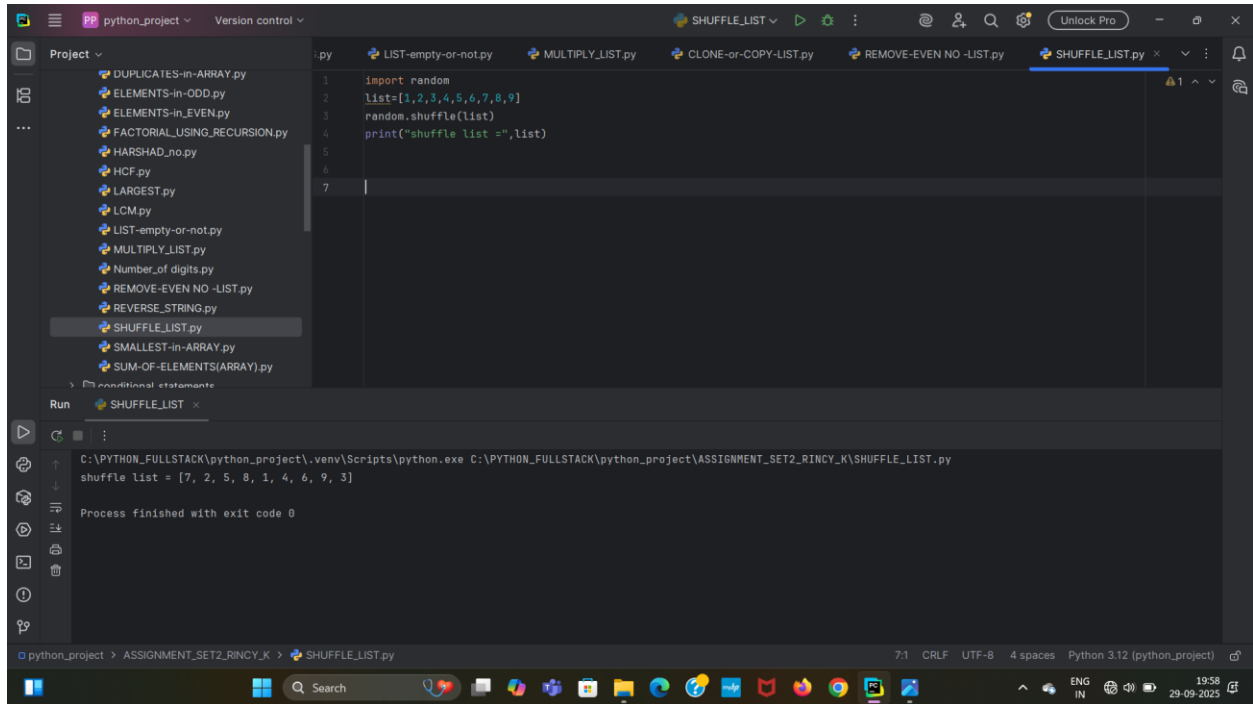
```
1 def remove_even_numbers(list): 1 usage
2     result=[]
3     for num in list:
4         if num % 2 != 0:
5             result.append(num)
6     return result
7 numbers=[1,2,3,4,5,6,7,8,9]
8 print(remove_even_numbers(numbers))
9
10
```

The Run and Debug console at the bottom shows the execution of the script:

```
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe "C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\REMOVE-EVEN NO -LIST.py"
[1, 3, 5, 7, 9]
Process finished with exit code 0
```

The status bar at the bottom indicates the file is 'REMOVE-EVEN NO -LIST.py' with 4:21 CRLF UTF-8 encoding, 4 spaces, and Python 3.12.

25.SHUFFLE

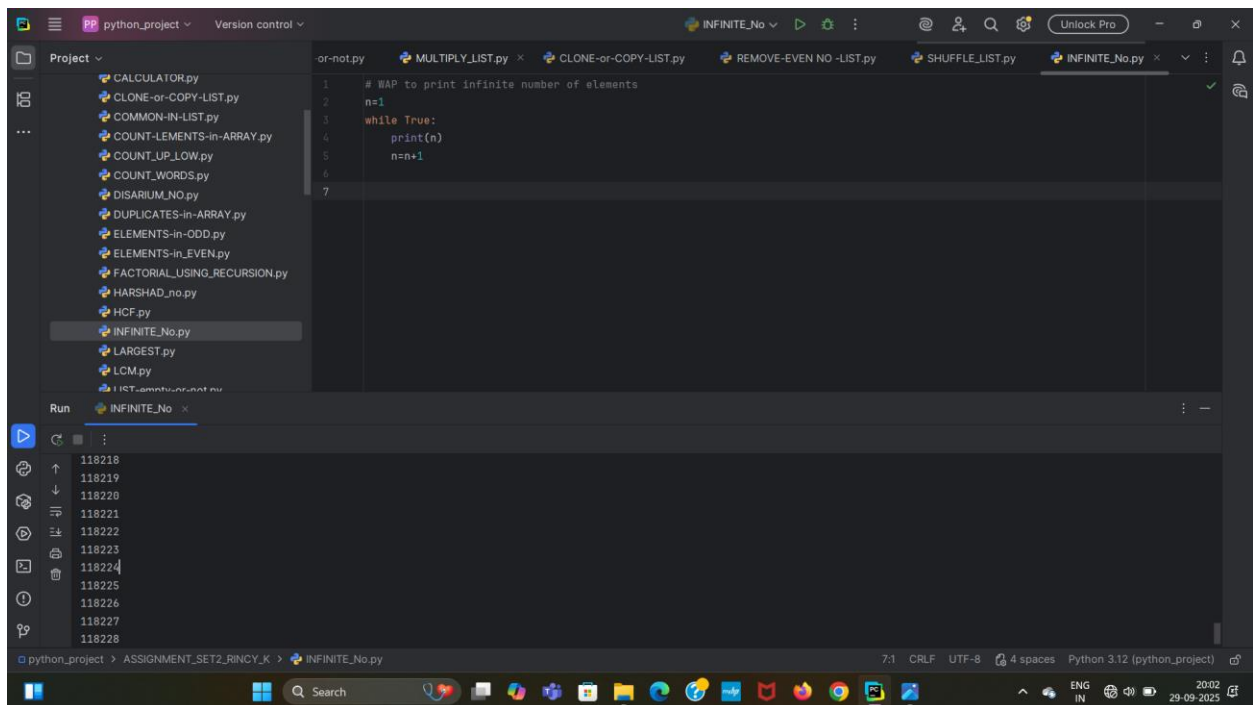


The screenshot shows a code editor with a project named 'python_project'. The file explorer on the left lists various Python files, including 'SHUFFLE_LIST.py'. The main editor displays the code for 'SHUFFLE_LIST.py':

```
1 import random
2 list=[1,2,3,4,5,6,7,8,9]
3 random.shuffle(list)
4 print("shuffle list =",list)
5
6
7
```

The Run console at the bottom shows the execution of the script, outputting 'shuffle list = [7, 2, 5, 8, 1, 4, 6, 9, 3]' and indicating that the process finished with exit code 0.

26.INFINITE No OF ELEMENTS

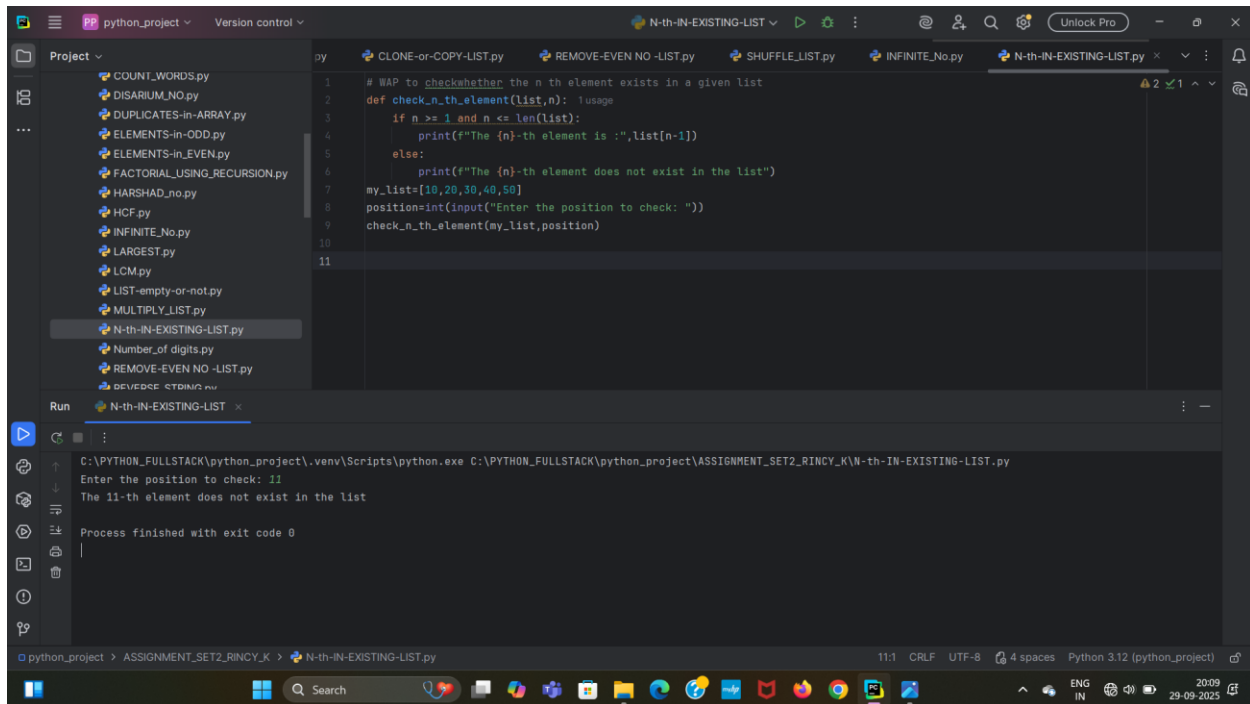


The screenshot shows a code editor with a project named 'python_project'. The file explorer on the left lists various Python files, including 'INFINITE_No.py'. The main editor displays the code for 'INFINITE_No.py':

```
1 # WAP to print infinite number of elements
2 n=1
3 while True:
4     print(n)
5     n=n+1
6
7
```

The Run console at the bottom shows the execution of the script, outputting a sequence of numbers from 118218 to 118228.

27. NTH IN EXISTING LIST



```
1 # WAP to check whether the n th element exists in a given list
2 def check_nth_element(list,n):
3     if n >= 1 and n <= len(list):
4         print(f"The {n}-th element is :",list[n-1])
5     else:
6         print(f"The {n}-th element does not exist in the list")
7 my_list=[10,20,30,40,50]
8 position=int(input("Enter the position to check: "))
9 check_nth_element(my_list,position)
10
11
```

Run N-th-IN-EXISTING-LIST

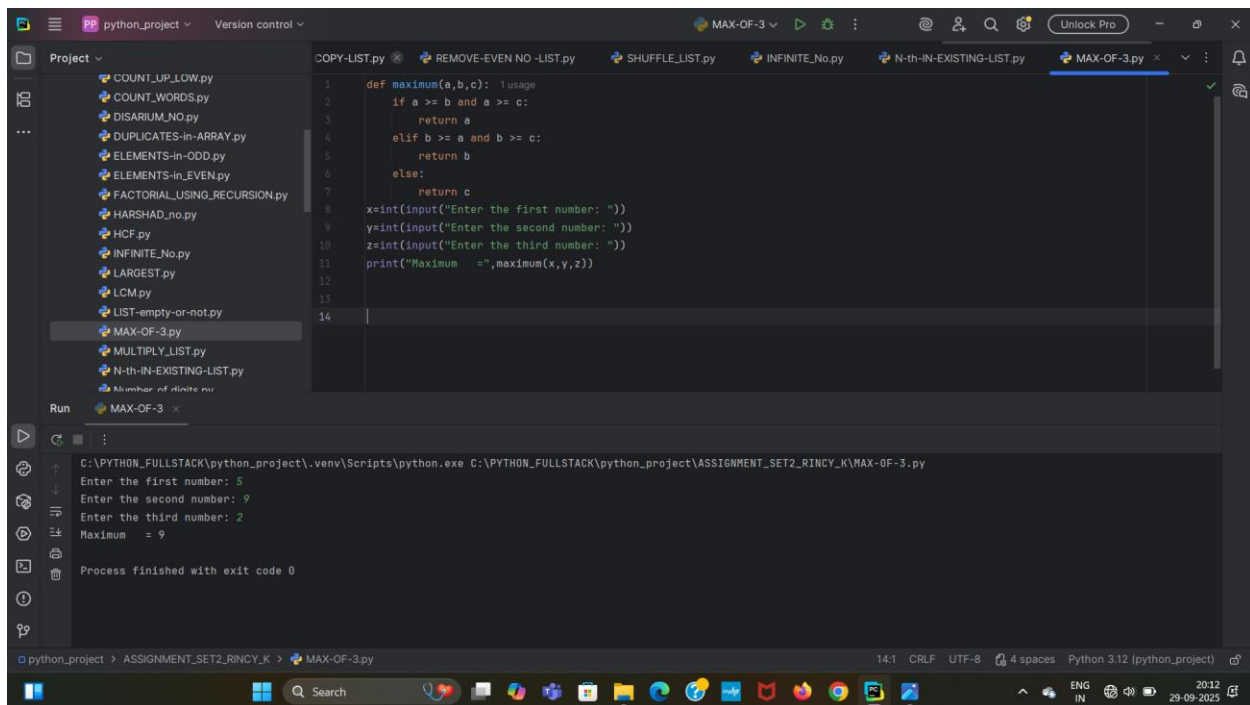
C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\N-th-IN-EXISTING-LIST.py

Enter the position to check: 11

The 11-th element does not exist in the list

Process finished with exit code 0

28. MAX OF THREE



```
1 def maximum(a,b,c):
2     if a >= b and a >= c:
3         return a
4     elif b >= a and b >= c:
5         return b
6     else:
7         return c
8 x=int(input("Enter the first number: "))
9 y=int(input("Enter the second number: "))
10 z=int(input("Enter the third number: "))
11 print("Maximum = ",maximum(x,y,z))
12
13
14
```

Run MAX-OF-3

C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\MAX-OF-3.py

Enter the first number: 5

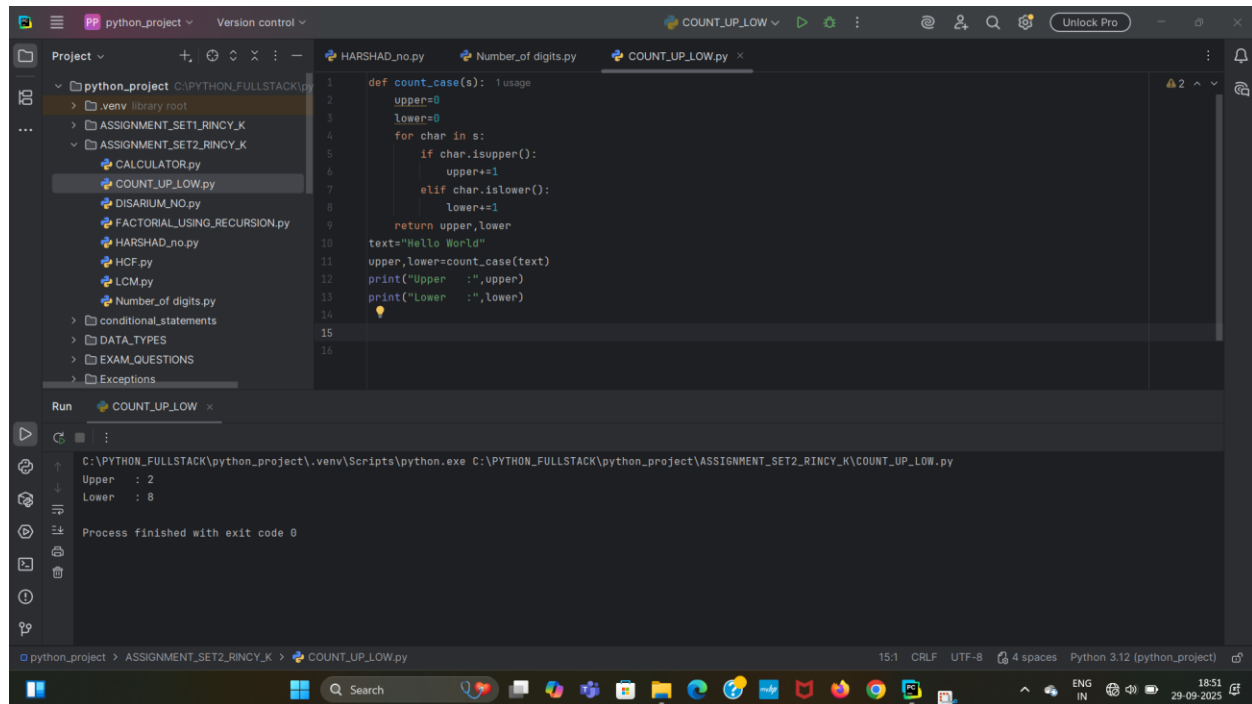
Enter the second number: 9

Enter the third number: 2

Maximum = 9

Process finished with exit code 0

29.COUNT UPPER CASE & LOWER CASE USING FUNCTION



The screenshot shows a code editor with a project named 'python_project'. The file 'COUNT_UP_LOW.py' is open, displaying a Python function 'count_case(s)' that iterates through a string 's' and counts the number of uppercase and lowercase letters. The function returns a tuple of counts. The main code calls this function with the string 'Hello World' and prints the results. The output in the console shows 'Upper : 2' and 'Lower : 8'. The process finished with exit code 0.

```
1 def count_case(s):
2     upper=0
3     lower=0
4     for char in s:
5         if char.isupper():
6             upper+=1
7         elif char.islower():
8             lower+=1
9     return upper,lower
10 text="Hello World"
11 upper,lower=count_case(text)
12 print("Upper  :",upper)
13 print("Lower  :",lower)
14
15
16
```

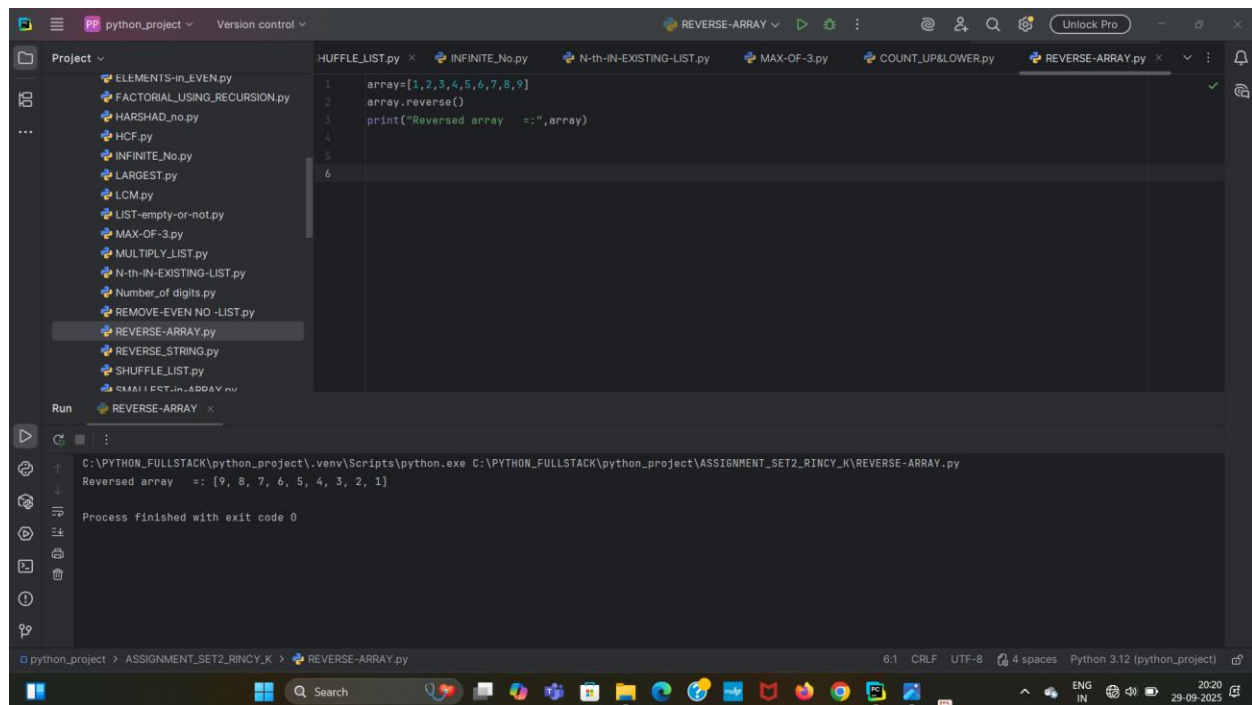
Run: COUNT_UP_LOW x

C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\COUNT_UP_LOW.py

Upper : 2
Lower : 8

Process finished with exit code 0

30.REVERSE ARRAY



The screenshot shows a code editor with a project named 'python_project'. The file 'REVERSE-ARRAY.py' is open, displaying a Python program that creates an array [1, 2, 3, 4, 5, 6, 7, 8, 9], reverses it using the 'reverse()' method, and prints the result. The output in the console shows 'Reversed array =: [9, 8, 7, 6, 5, 4, 3, 2, 1]'. The process finished with exit code 0.

```
1 array=[1,2,3,4,5,6,7,8,9]
2 array.reverse()
3 print("Reversed array  =:",array)
4
5
6
```

Run: REVERSE-ARRAY x

C:\PYTHON_FULLSTACK\python_project\.venv\Scripts\python.exe C:\PYTHON_FULLSTACK\python_project\ASSIGNMENT_SET2_RINCY_K\REVERSE-ARRAY.py

Reversed array =: [9, 8, 7, 6, 5, 4, 3, 2, 1]

Process finished with exit code 0